Verilog 第一次实验报告

实验名称	基于 Vivado 的加法器设计与仿真				
学生姓名	赵宇哲	学号	2411094	指导老师	杨飞
实验地点	公共实验楼 A 区 306 教室		实验时间	2025-9-24 星期三第三四节	

1 实验项目名称

基于 Vivado 的加法器设计与仿真

2 实验平台

Xilinx Vivado Design Suite 2017.1（64-bit）

3 关键问题

3.1 assign & always 区别

- assign: 左边必须是 wire，只能写在模块体内、always/initial 块之外。同时是“连续赋值”（类似引用），左右值会实时变化更新。
- always: 左边通常是 reg，只能在模块体内使用。同时是“过程赋值”（类似传参），只有在条件触发代码运行时才会赋值。

3.2 reg & wire 区别

- reg
  - 在设计文件中：由 assign 操作，且不能存值，起到连接功能（像导线），因此可以实时变化
  - 在仿真文件中：接被测模块的输出端信号
- wire
  - 在设计文件中：表示过程块（always / initial）里赋值的信号，可以存储值，但触发前会一直保持上次值，不会实时更新
  - 在仿真文件中：被测模块的输入端信号

3.2.1 PPT-P3

a[0]	a[1]	b[0]	b[1]	equal	a[0]	a[1]	b[0]	b[1]	equal
0	0	0	0	1	1	0	0	0	0
0	0	0	1	0	1	0	0	1	0
0	0	1	0	0	1	0	1	0	1
0	0	1	1	0	1	0	1	1	0
0	1	0	0	0	1	1	0	0	0
0	1	0	1	1	1	1	0	1	0
0	1	1	0	0	1	1	1	0	0
0	1	1	1	0	1	1	1	1	1

表 1: 2 位比较器的真值表

4 实验内容

4.1 半加器

半加器代码与仿真

4.1.1 设计代码

```
module half_add(
    input wire a,
    input wire b,
    output wire sum,
    output wire carry
    );
assign sum = a ^ b;
assign carry = a & b;
endmodule
```

4.1.2 仿真代码

```
`timescale 1ns / 1ps
module tb_half_add();
    reg a, b;
    wire sum, carry;


    half_add uut(
        .a(a),
        .b(b),
        .sum(sum),
        .carry(carry)
    );

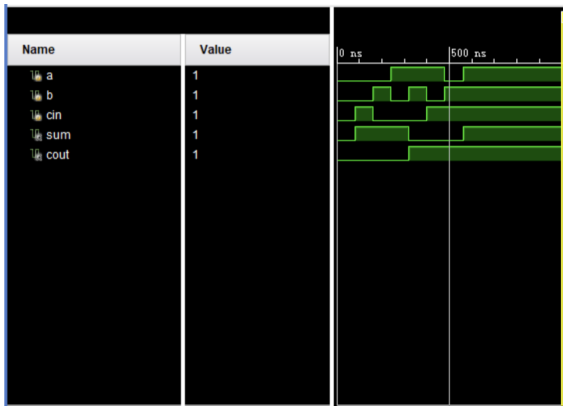
    initial begin

            a=0; b=0;  #200;
            a=1; b=0;  #200;
            a=0; b=1;  #200;
            a=1; b=1;  #200;

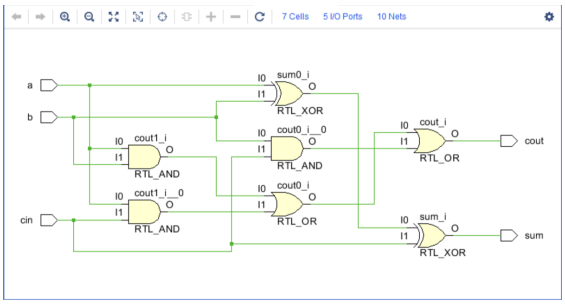
    end

    initial begin
        $monitor("时刻=%0t, a=%b, b=%b, sum=%b, carry=%b",
                 $time, a, b, sum, carry);
    end
endmodule
```

4.1.3 仿真结果



半加器波形图



半加器电路

4.1.4 流程分析

通过第一次实验我们已经知道，半加器可以看作由一个异或门和一个与门组成，负责对两个一位二进制数进行加法运算。输入为 a 、b ，输出包括“和” sum 和“进位” $carry$ ，其中 sum 由 $a \oplus b$ 得到，$carry$ 由 $a\&b$ 得到。半加器没有进位输入，只能处理一位的加法，不用考虑来自低位的进位信号。

4.2 全加器

全加器代码与仿真

4.2.1 设计代码

```
module full_add(
    input a,
    input b,
    input cin,
    output sum,
    output cout
);
assign sum = a ^ b ^ cin;
assign cout = (a & b) | (a & cin) | (b & cin);
endmodule
```

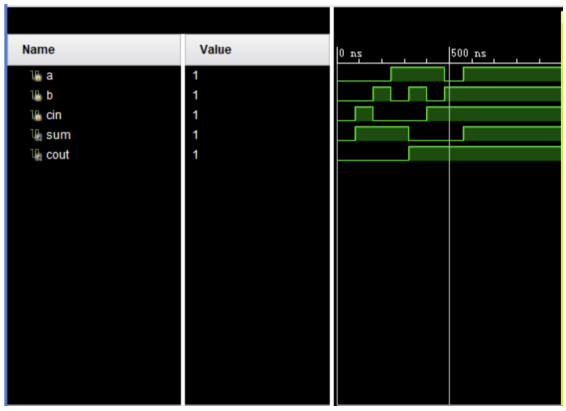
4.2.2 仿真代码

```
`timescale 1ns / 1ps
module td_full_add();
    reg a, b, cin;
    wire sum, cout;
    full_add uut(
        .a(a),
        .b(b),
        .cin(cin),
        .sum(sum),
        .cout(cout)
    );

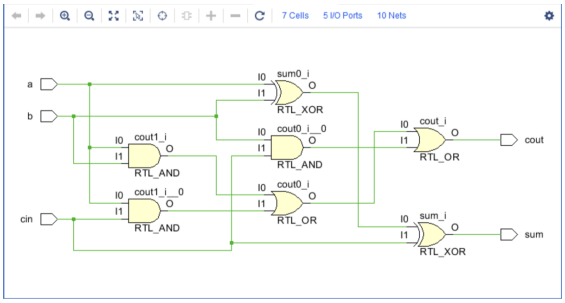
    initial begin
        // 每种组合 50ns
        a=0; b=0; cin=0; #80;
        a=0; b=0; cin=1; #80;
        a=0; b=1; cin=0; #80;
        a=1; b=0; cin=0; #80;
        a=1; b=1; cin=0; #80;
        a=1; b=0; cin=1; #80;
        a=0; b=1; cin=1; #80;
        a=1; b=1; cin=1; #80;
    end

    initial begin
        $monitor("时刻=%0t, a=%b, b=%b, cin=%b, sum=%b, cout=%b",
                 $time, a, b, cin, sum, cout);
    end
endmodule
```

4.2.3 仿真结果



全加器波形图



全加器电路

4.2.4 流程分析

全加器可以看作是在半加器的基础上增加了对进位输入的处理。它有三个输入 a 、b 和进位输入 cin，输出包括“和” sum 和“进位” cout，其中 sum 由 $a \wedge b \wedge \text{cin}$ 得到，cout 由 $(a\&b)|(a\&\text{cin})|(b\&\text{cin})$ 得到。全加器能够在一位加法的基础上同时处理来自低位的进位输入，并输出进位 cout 到高一位。

4.3 8 位加法器

8 位加法器代码与仿真

4.3.1 设计代码

```
//全加器
module full_adder(
  input  wire a, b, cin,
  output wire sum, cout
);
  assign sum  = a ^ b ^ cin;
  assign cout = (a & b) | (cin & (a ^ b));
endmodule

//级联8个全加器
module adder8(
  input  wire [7:0] a,
  input  wire [7:0] b,
  input  wire       cin,
  output wire [7:0] sum,
  output wire       cout
);
  wire [8:0] c;
  assign c[0] = cin;

  genvar i;
  generate
    for (i = 0; i < 8; i = i + 1) begin : FA
      full_adder u_fa (
        .a   (a[i]),
        .b   (b[i]),
        .cin (c[i]),
        .sum (sum[i]),
        .cout(c[i+1])
      );
    end
  endgenerate

  assign cout = c[8];
endmodule
```

4.3.2 仿真代码

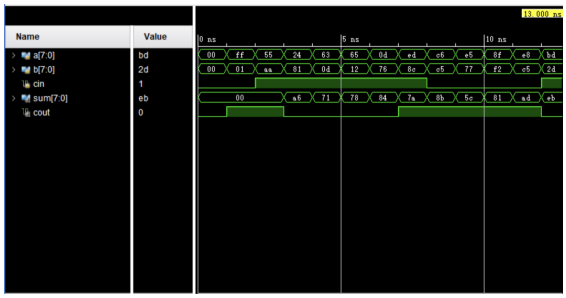
```
`timescale 1ns/1ps
module tb_adder8;
  reg  [7:0] a, b;
  reg        cin;
  wire [7:0] sum;
  wire       cout;

  adder8 dut(.a(a), .b(b), .cin(cin), .sum(sum), .cout(
      cout));

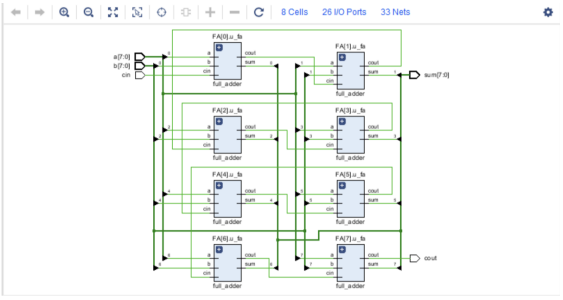
  initial begin
    // 几组固定用例
    a=8'h00; b=8'h00; cin=0; #1 $display("0) %h + %h + %b
      = {%b,%h}", a,b,cin,cout,sum);
    a=8'hFF; b=8'h01; cin=0; #1 $display("1) %h + %h + %b
      = {%b,%h}", a,b,cin,cout,sum);
    a=8'h55; b=8'hAA; cin=1; #1 $display("2) %h + %h + %b
      = {%b,%h}", a,b,cin,cout,sum);

    // 随机测试
    repeat (10) begin
      a   = $urandom;
      b   = $urandom;
      cin = $urandom % 2;
      #1;
      // 参考模型对比
      if ({cout,sum} !== (a + b + cin)) begin
        $display("MISMATCH: a=%h b=%h cin=%b -> got {%b,%h
      }, expect %h",
                 a,b,cin,cout,sum, a+b+cin);
        $stop;
      end
    end
    $display("All tests passed.");
    $finish;
  end
endmodule
```

4.3.3 仿真结果



8 位加法器波形图



8 位加法器电路

4.3.4 流程分析

8 位加法器可以看作 8 个全加器的级联。最低位全加器的 cin 由外部输入提供，其产生的进位输出 c[1] 作为下一位全加器的 cin 输入，依次传递。最高位全加器的 c[8] 作为 8 位加法器的 cout 输出。